

Pad-Based Crypton

Look Ups using an Infinite Pad

Three Encryption Inputs

- **Plaintext** - A Bunch of Non-Random Bytes
 - AA-BB-CC-DD-EE-FF
- **Symmetric Key** - Bytes Needed by Symmetric Encryption Algorithm
 - Key, IV and Signature (for Aegis)
- **Encryption Pad** - A Pseudo Random Bunch of Bytes
 - AB-CD-EF-FE-DC-BA-AB-CD-EF-FE-DC-BA

Randomize Pad and Plaintext

Plaintext => Symmetric Algorithm => Randomized Plaintext

AA-BB-CC-DD-EE-FF becomes 0A-B5-C4-3D-ED-4F

Pad => Symmetric Algorithm => Randomized Pad

AB-CD-EF-FE-DC-BA-AB-CD-EF-FE-DC-BA
becomes

E3-C4-0A-F7-C4-B5-AB-CD-2F-C4-D7-4A

Look Up Randomized Plaintext

Randomized Plaintext 0A-B5-C4-3D-ED-4F

2

5

9

?!

Randomized Pad E3-C4-0A-F7-C4-B5-AB-CD-2F-C4-D7-4A

0

1

2

3

4

5

6

7

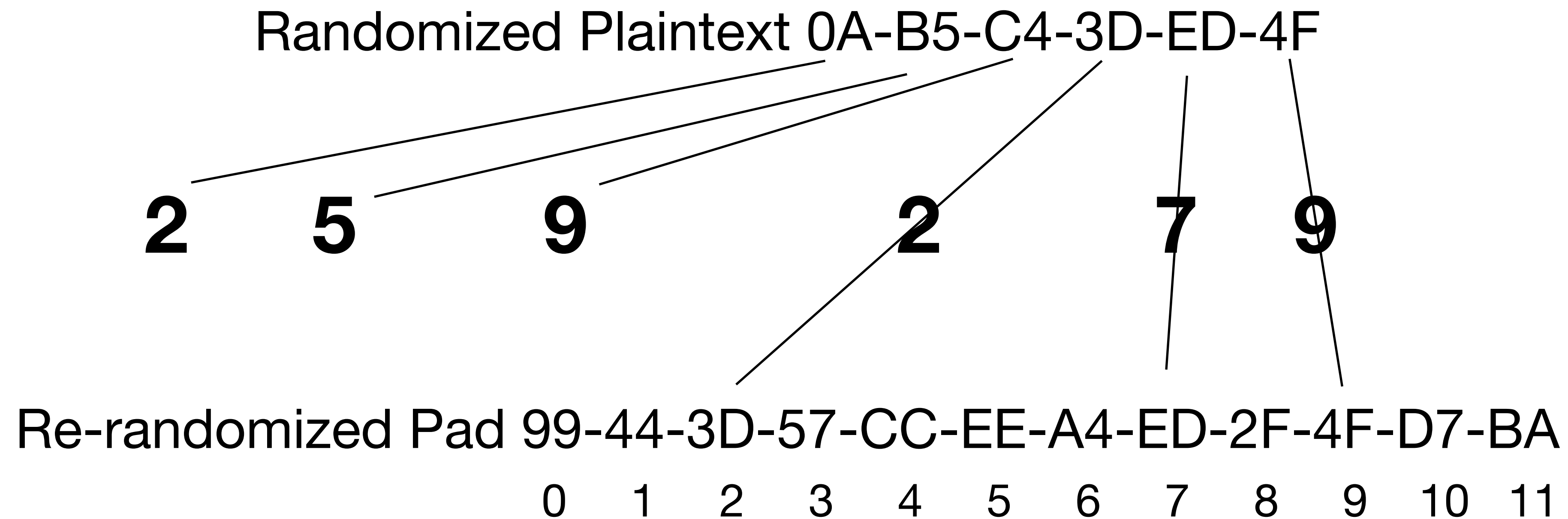
8

9

10

11

Randomize Pad Again



Look Ups Are:

2 5 9 2 7 9

or 02-05-09-02-07-09 in obviously numeric bytes

Randomize Look Ups

Look Ups => Symmetric Algorithm => Randomized Cyphertext

02-05-09-02-07-09 becomes 35-85-4C-BB-24-E5

35-85-4C-BB-24-E5 is the Encrypted Cyphertext

Three Decryption Inputs

- **Cyphertext** - The Encrypted Look Ups
 - 35-85-4C-BB-24-E5
- **Symmetric Key** - Bytes Needed by Symmetric Encryption Algorithm
 - Key, IV and Signature (for Aegis)
- **Encryption Pad** - A Pseudo Random Bunch of Bytes
 - AB-CD-EF-FE-DC-BA-AB-CD-EF-FE-DC-BA

Decrypt Cyphertext

Cyphertext => Symmetric Algorithm => Look Ups

35-85-4C-BB-24-E5 becomes 02-05-09-02-07-09

which are the Look Ups: 2 5 9 2 7 9

Randomize Pad

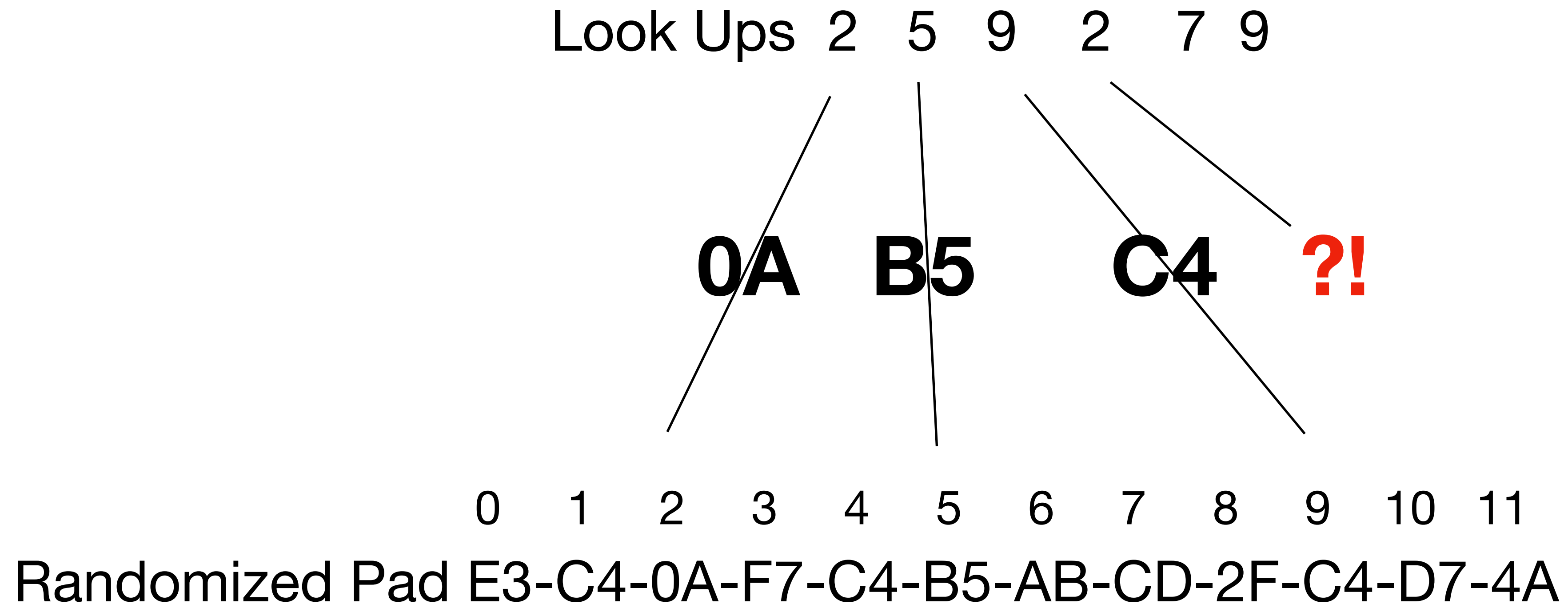
Pad => Symmetric Algorithm => Randomized Pad

AB-CD-EF-FE-DC-BA-AB-CD-EF-FE-DC-BA

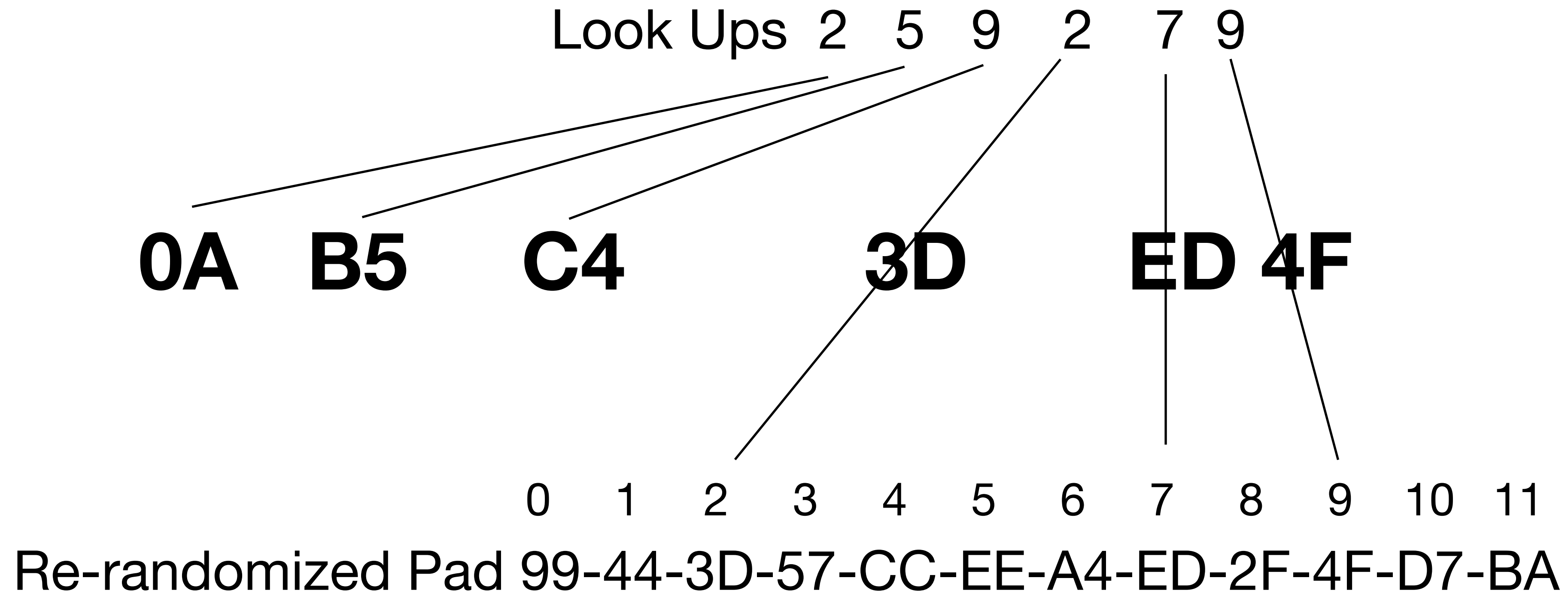
becomes

E3-C4-0A-F7-C4-B5-AB-CD-2F-C4-D7-4A

Find Look Ups in the Pad



Randomize Pad Again



Recovered Bytes Are:

0A B5 C4 3D ED 4F

which was the Randomized Plaintext

Decrypt Randomized Plaintext

Randomized Plaintext => Symmetric Algorithm => Plaintext

0A-B5-C4-3D-ED-4F becomes AA-BB-CC-DD-EE-FF

AA-BB-CC-DD-EE-FF is the Decrypted Plaintext

Practical Key Generation

- Start with Passphrase and Secret Timestamp
- $\text{Current Timestamp} - \text{Secret Timestamp} = \text{Offset}$
- $\text{Current Timestamp} - 2 \times \text{Offset} = \text{Encryption Timestamp}$
- Encryption Timestamp converted to words = Other Passphrase
- Passphrase plus Other Passphrase = One Time Passphrase
- One Time Passphrase into Argon2 = Symmetric Key

Practical Pad Generation 1

- Start with Unique Image from Camera in “normal” format
- Unique Image in PNG Image Format on USB Drive = Stored Pad
- Destroy Unique Image from Camera in “normal” format
- Share copies of Stored Pad with other parties as needed
- RGB Bytes from Image Pixels in Stored Pad = Encryption Pad
- Destroy each copy of Stored Pad (or whole USB Drive) after use

Practical Pad Generation 2

- Start with Passphrase and well documented song, poem or scripture Verse
- Share Passphrase and name of Verse with other parties as needed
- Looked up words from Verse with whitespace removed = Contents of Verse
- Passphrase into Argon2 = Symmetric Key
- Contents of Verse and Symmetric Key into Symmetric Algorithm = Pad Bytes
- Pad Bytes converted to PNG Image Format on USB Drive = Stored Pad
- RGB Bytes from Image Pixels in Stored Pad = Encryption Pad
- Destroy each copy of Stored Pad (or whole USB Drive) after use